

Exercicios de Treino PAW - Por Fases

Tenta resolver cada exercicio ANTES de consultar a seccao de Solucoes no final do documento.

FASE 1 - Fundamentos (HTML, CSS, Web)

Escolha Multipla

1.1 No modelo cliente-servidor, quem inicia a comunicacao?

- a) O servidor
- b) A base de dados
- c) O cliente (browser)
- d) O middleware

1.2 Qual destes NAO e um metodo HTTP valido?

- a) GET
- b) POST
- c) CREATE
- d) DELETE

1.3 Qual o codigo de status HTTP correto para "utilizador nao autenticado"?

- a) 400 Bad Request
- b) 401 Unauthorized
- c) 403 Forbidden
- d) 404 Not Found

1.4 Qual das seguintes afirmacoes e verdadeira?

- a) HTML e uma linguagem de programacao
- b) CSS e uma linguagem de estilos
- c) Node.js e uma linguagem de programacao
- d) MongoDB e uma base de dados relacional

1.5 Qual e o metodo HTTP idempotente usado para atualizar dados?

- a) POST
- b) PUT
- c) PATCH
- d) CREATE

1.6 (EXTRA) O que se encontra dentro da tag <head> de um documento HTML?

- a) Conteudo visivel da pagina
- b) Metadados como charset, title e links CSS
- c) Formularios de input
- d) Scripts que manipulam o DOM

1.7 (EXTRA) Qual destes elementos HTML e inline?

- a) <div>
- b) <p>
- c)
- d) <table>

Verdadeiro ou Falso

1.8 "HTML é uma linguagem de programação porque permite criar páginas web."

1.9 "Um formulário HTML pode submeter dados ao servidor sem JavaScript."

1.10 "É seguro usar o método GET para enviar passwords num formulário de login."

1.11 "O código HTTP 403 Forbidden indica que o utilizador não está autenticado."

1.12 (EXTRA) "A tag requer uma tag de fecho para ser válida."

1.13 (EXTRA) "No Box Model do CSS, o padding está entre o content e o border."

Resposta Aberta

1.14 Explica o modelo cliente-servidor na web, incluindo os papéis de cada parte e o fluxo típico de comunicação. (1-1.5 val)

1.15 Indica as 3 formas de adicionar CSS a uma página HTML. Qual é a recomendada e porque? (1-1.5 val)

FASE 2 - JavaScript & DOM

Escolha Múltipla

2.1 Qual é a diferença entre == e === em JavaScript?

- a) Não há diferença
- b) == compara valor e tipo, === só compara valor
- c) === compara valor e tipo, == compara só valor (com coerção)
- d) == é para strings, === é para números

2.2 O que retorna `document.getElementById("xyz")`?

- a) Todos os elementos com id "xyz"
- b) Uma coleção de elementos
- c) Um único elemento (o primeiro)
- d) null, porque IDs não existem

2.3 Qual é a diferença entre `localStorage` e `sessionStorage`?

- a) Não há diferença
- b) `localStorage` persiste permanentemente; `sessionStorage` dura apenas a sessão/tab
- c) `sessionStorage` persiste permanentemente; `localStorage` dura apenas a sessão
- d) Ambos persistem permanentemente

2.4 Qual a diferença entre `var`, `let` e `const`?

- a) São exatamente iguais
- b) `var` tem scope de função + hoisting; `let` tem scope de bloco; `const` é constante
- c) `let` tem scope de função; `var` tem scope de bloco
- d) `const` pode ser reatribuído após a declaração

2.5 O que faz o AJAX?

- a) Recarrega toda a página para atualizar conteúdo
- b) Permite atualizar partes da página sem recarregar toda a página
- c) É uma linguagem de programação
- d) É um servidor web

2.6 (EXTRA) Qual dos seguintes estados NÃO é um estado válido de uma Promise?

- a) pending
- b) fulfilled
- c) loading
- d) rejected

2.7 (EXTRA) Qual é a diferença entre `innerHTML` e `textContent`?

- a) São iguais
- b) `innerHTML` interpreta HTML; `textContent` trata como texto puro
- c) `textContent` interpreta HTML; `innerHTML` trata como texto puro
- d) Ambos interpretam HTML

Verdadeiro ou Falso

2.8 "Em JavaScript, `var` sofre hoisting mas `let` e `const` não."

2.9 "AJAX requer sempre que a página inteira seja recarregada para funcionar."

2.10 "`JSON.parse()` converte um objeto JavaScript numa string JSON."

2.11 (EXTRA) "Arrow functions e function declarations comportam-se de forma idêntica em todos os contextos."

2.12 (EXTRA) "O `readyState == 4` numa chamada `XMLHttpRequest` indica que a resposta esta completa."

Resposta Aberta

2.13 O seguinte codigo HTML/JS tem 3 erros. Identifica e corrige cada um. (1-2 val)

```
<html>
<body>
  <h1>Click Counter</h1>
  <p>Click Number: <span id="contador">0</span></p>
  <button>Click Me</button>
  <script>
    function contarClique() {
      numeroCliques++;
      document.getElementById(counter).textContent = numeroCliques;
    }
  </script>
</body>
</html>
```

2.14 Explica como usar `localStorage` para manter dados entre recarregamentos de pagina. (1 val)

FASE 3 - Backend: Node.js, Express, BD & REST

Escolha Multipla

3.1 O que e o Node.js?

- a) Uma linguagem de programacao
- b) Um framework web
- c) Um runtime JavaScript para executar JS fora do browser
- d) Um servidor web pronto a usar

3.2 Qual a diferenca entre `req.params`, `req.query` e `req.body`?

- a) Sao todos iguais
- b) `req.params` vem da rota (`:id`), `req.query` vem da query string (`?key=val`), `req.body` vem do corpo do pedido
- c) `req.body` vem do URL, `req.params` vem do corpo do pedido
- d) `req.query` vem do corpo do pedido POST

3.3 O que o NPM NAO faz?

- a) Instalar packages/modulos
- b) Gerir dependencias
- c) Criar componentes Angular
- d) Executar scripts (npm start)

3.4 O que e middleware no Express?

- a) Uma base de dados
- b) Funcoes executadas entre o pedido e a resposta com acesso a req, res e next()
- c) O template engine EJS
- d) O motor de rendering do Angular

3.5 Qual tag EJS produz output HTML escaped?

- a) `<% %>`
- b) `<%= %>`
- c) `<%- %>`
- d) `<!-- %>`

3.6 (EXTRA) No padrao MVC aplicado a Express, onde ficam os schemas/models Mongoose?

- a) Na pasta views/
- b) Na pasta controllers/
- c) Na pasta models/
- d) Na pasta public/

3.7 (EXTRA) O que distingue uma BD relacional (SQL) de uma BD NoSQL como o MongoDB?

- a) SQL nao suporta transacoes
- b) SQL tem schema rigido e bem definido; NoSQL e flexivel e sem schema obrigatorio
- c) NoSQL so guarda dados em formato XML
- d) SQL escala melhor horizontalmente

3.8 (EXTRA) Qual e o middleware de erros correto no Express?

- a) `app.use(function(req, res, next) { ... })`
- b) `app.use(function(err, req, res, next) { ... })`
- c) `app.error(function(err) { ... })`
- d) `app.catch(function(err, res) { ... })`

Verdadeiro ou Falso

3.9 "Express e um framework que executa no browser do utilizador."

3.10 "REST APIs comunicam APENAS em JSON."

3.11 "O Mongoose e uma base de dados NoSQL."

3.12 "JWT (JSON Web Token) obriga a usar uma REST API."

3.13 "A ordem do middleware no Express e irrelevante."

3.14 (EXTRA) "O NPM pode ser usado para instalar o MongoDB no sistema."

3.15 (EXTRA) "A leitura bloqueante (sincrona) com `fs.readFileSync()` e a abordagem recomendada em Node.js."

Resposta Aberta

3.16 Explica o padrao MVC e como se aplica numa aplicacao Express com MongoDB. (1.5 val)

3.17 Explica a diferenca entre autenticacao e autorizacao. Indica os codigos HTTP associados e da exemplos praticos. (1.5 val)

3.18 Descreve as propriedades de uma REST API e como implementarias uma operacao CRUD completa em Express. (2 val)

FASE 4 - Angular & Full-Stack

Escolha Multipla

4.1 Qual e a linguagem usada no Angular?

- a) JavaScript
- b) Java
- c) TypeScript
- d) CoffeeScript

4.2 Qual o comando para criar um novo projeto Angular?

- a) `npm new my-app`
- b) `ng new my-app`
- c) `angular create my-app`
- d) `npm create angular my-app`

4.3 Em Angular, qual sintaxe implementa two-way data binding?

- a) {{ variavel }}
- b) [propriedade]="valor"
- c) (evento)="handler()"
- d) [(ngModel)]="variavel"

4.4 O que é @Injectable() em Angular?

- a) Um template HTML
- b) Um decorador que permite injetar a classe via Dependency Injection
- c) Um modulo de routing
- d) Uma diretiva de formatacao

4.5 Qual a diferenca entre Guard e Interceptor em Angular?

- a) Sao iguais
- b) Guard protege rotas (CanActivate); Interceptor modifica pedidos HTTP (HttpInterceptor)
- c) Interceptor protege rotas; Guard modifica pedidos
- d) Ambos sao template engines

4.6 Qual hook do ciclo de vida Angular é o mais usado e executado uma unica vez apos inicializacao?

- a) ngOnChanges()
- b) ngOnInit()
- c) ngOnDestroy()
- d) ngAfterViewInit()

4.7 (EXTRA) O que é uma SPA (Single Page Application)?

- a) Uma aplicacao que usa apenas uma unica tag HTML
- b) Uma aplicacao com uma unica pagina que atualiza conteudo dinamicamente sem recarregar
- c) Uma aplicacao sem CSS
- d) Uma aplicacao que nao usa JavaScript

4.8 (EXTRA) Como é que um componente filho recebe dados do componente pai em Angular?

- a) Via @Output()
- b) Via @Input()
- c) Via localStorage
- d) Via req.body

Verdadeiro ou Falso

4.9 "Angular é um framework fullstack que executa tanto no frontend como no backend."

4.10 "Em Angular, a rota de login deve ter canActivate: [AuthGuard] para ser segura."

4.11 "TypeScript é um superset de JavaScript - todo código JS válido é também TS válido."

4.12 "Serviços Angular podem ser injetados em múltiplos componentes."

4.13 (EXTRA) "O ng build gera o build de producao na pasta node_modules/."

4.14 (EXTRA) "Um Observable em Angular retorna o valor imediatamente de forma sincrona."

Resposta Aberta

4.15 Explica a diferenca entre Guards e Interceptors em Angular. Da exemplos de codigo de cada um. (1.5 val)

4.16 Como funciona um servico Angular? Explica o que sao @Injectable(), HttpClient e Observable. (1.5 val)

FASE 5 - Seguranca & Identificacao de Tecnologia

Escolha Multipla

5.1 Porque e que a validacao deve ser feita no frontend E no backend?

- a) Porque o frontend e mais seguro que o backend
- b) Frontend para UX (feedback rapido), backend para seguranca (utilizador pode contornar frontend)
- c) Porque o backend nao consegue validar dados
- d) Porque a validacao so funciona no frontend

5.2 Qual excerto de codigo identifica um servico Angular?

- a) req, res, next, app.use()
- b) @Injectable(), Observable, HttpClient
- c) mongoose.Schema, mongoose.model
- d) document.getElementById(), addEventListener

5.3 Qual das seguintes afirmacoes sobre seguranca e FALSA?

- a) Credenciais nunca devem estar no codigo frontend
- b) Passwords devem ser guardadas em formato hashed
- c) Validacao apenas no frontend e suficiente para seguranca
- d) HTTPS deve ser usado em producao

5.4 (EXTRA) Que tecnologia usarias se visses <%= %> e <% %> num ficheiro?

- a) Angular Component

- b) React JSX
- c) EJS (Template Engine)
- d) Vue.js

5.5 (EXTRA) Qual o principal risco de usar innerHTML em vez de textContent?

- a) Performance mais lenta
- b) Injecao de codigo HTML/JavaScript malicioso (XSS)
- c) Nao funciona em todos os browsers
- d) Nao ha risco

Verdadeiro ou Falso

5.6 "MongoDB pode correr diretamente no frontend (browser)."

5.7 "JavaScript funciona apenas no frontend (browser)."

5.8 "Um servico Angular pode ter um template HTML associado."

5.9 (EXTRA) "CORS (Cross-Origin Resource Sharing) e uma restricao imposta pelo servidor."

5.10 (EXTRA) "O Swagger/OpenAPI serve para executar APIs REST automaticamente."

Resposta Aberta

5.11 Explica porque e que a validacao de dados deve ser feita no frontend E no backend. Da 3 razoes concretas para validar no backend. (2-3 val)

5.12 Dado o seguinte excerto de codigo (AuthInterceptor), identifica a tecnologia usada e explica o que o codigo faz: (1.5 val)

```
@Injectable({ providedIn: 'root' })
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const token = localStorage.getItem('token');
    const cloned = req.clone({
      headers: req.headers.set('Authorization', 'Bearer ' + token)
    });
    return next.handle(cloned);
  }
}
```

Exercicios Integrados (Mistura de Fases)

Estes exercicios simulam o formato real do exame, combinando multiplos temas.

Escolha Multipla

- I.1** Qual das seguintes afirmacoes sobre o express-generator e verdadeira?
- a) Cria componentes Angular automaticamente
 - b) Cria uma estrutura de projeto Express com pasta views para templates
 - c) E um template engine como EJS
 - d) Instala o MongoDB automaticamente
- I.2** Qual destas combinacoes de tecnologia esta errada?
- a) Angular - Frontend no browser
 - b) Express - Backend no servidor
 - c) Mongoose - Template engine
 - d) JWT - Autenticacao por tokens
- I.3** Em qual situacao usarias `res.render()` vs `res.json()`?
- a) Sao iguais
 - b) `res.render()` para server-side rendering com templates; `res.json()` para enviar dados JSON a uma API REST
 - c) `res.json()` para templates; `res.render()` para APIs
 - d) Ambos enviam HTML

Verdadeiro ou Falso

- I.4** "O Angular pode aceder diretamente ao MongoDB usando Mongoose no browser."

- I.5** "Node.js e single-threaded com I/O bloqueante."

- I.6** "O npm start compila TypeScript e instala MongoDB."

Resposta Aberta

- I.7** Uma aplicacao web tem um formulario de registo. Descreve todas as camadas de seguranca e validacao que implementarias, desde o frontend ate a base de dados. (3 val)

- I.8** Corrige o seguinte codigo de um servidor Express que tem varios erros: (2 val)

```
const express = require('express');
const app = express();

app.get('/api/produto/:id', (req, res) => {
  const id = req.query.id;
  const produto = Produto.find(id);
  res.send(produto);
});

app.use(express.json());

app.post('/login', (req, res) => {
  const password = req.params.password;
  res.status(200);
});

app.listen(3000);
```

Resumo dos Exercicios

Fase	Escolha Multipla	V/F	Resposta Aberta	Total
Fase 1 - Fundamentos	7	6	3	16
Fase 2 - JavaScript & DOM	7	5	4	16
Fase 3 - Backend & REST	8	7	5	20
Fase 4 - Angular & Full-Stack	8	6	4	18
Fase 5 - Seguranca	5	5	4	14
Integrados	3	3	2	8
Exames Reais	23	10	14	47
TOTAL	61	42	36	139 exercicios

Dica: Comeca pela Fase 1 e avanca sequencialmente. Faz os exercicios SEM consultar as solucoes. As solucoes estao na segunda parte do documento.

EXAMES REAIS - Perguntas de Anos Anteriores

As perguntas que se seguem sao reais dos exames de Epoca Normal de 2020/21, 2021/22 e 2024/25.

EXAME 2020/2021 - Epoca Normal

Duracao: 1h30min | Parte 1: 7 valores | Parte 2: 3 valores | Parte 3: 10 valores

Parte 1 - Escolha Multipla (7 valores)

Cada pergunta vale 1 valor. Opcao incorreta desconta 0.5 valores.

P1. Indique as afirmacoes verdadeiras:

- a. CSS e uma linguagem de programacao para a web;
- b. Numa pagina da internet escrita em HTML, a informacao escrita entre as tags <head>...</head> nao e visivel na pagina apresentada no browser;
- c. Podemos submeter informacao para um servidor web sem utilizar JavaScript numa pagina escrita em HTML;
- d. O uso de CSS ou JavaScript nao e obrigatorio numa pagina HTML.

P2. Entre as seguintes opcoes escolha os metodos HTTP que podem ser utilizados para enviar dados a um servidor numa API REST:

- a. GET
- b. DELETE
- c. CREATE
- d. UPDATE

P3. Considerando a framework ExpressJS utilizada em Node.js, indique as afirmacoes verdadeiras:

- a. Podemos criar componentes com TypeScript e o padrao MVC;
- b. Podemos utilizar template engines para gerar paginas HTML dinamicamente no servidor;
- c. Podemos aceder diretamente a uma base de dados para enviar e guardar informacao com o modulo mongoose;
- d. Podemos executar funcoes JavaScript da framework ExpressJS diretamente a partir de um browser de internet.

P4. Da seguinte lista selecione as afirmacoes verdadeiras sobre uma aplicacao escrita com a framework Angular:

- a. Devemos manter as ligacoes diretas a uma base de dados (ex: MongoDB) para persistir informacao;
- b. Podemos utilizar ficheiros de texto para guardar informacao da aplicacao;
- c. Podemos utilizar a ferramenta npm para gerir dependencias da aplicacao;
- d. Podemos dividir as paginas de internet em componentes reutilizaveis;

P5. Indique as afirmacoes verdadeiras sobre o desenvolvimento de servicos REST:

- a. Podemos usar o modulo swagger para documentar e testar APIs REST;
- b. Os servicos REST enviam paginas HTML para um cliente;
- c. As APIs REST nao respeitam a arquitetura cliente-servidor na maioria das aplicacoes web;
- d. E possivel utilizar tokens JWT para garantir autenticao e autorizacao na API REST.

P6. Observe a figura (authController.verifyToken com jwt.verify) e indique as afirmacoes verdadeiras:

- a. Esta a ser validado um token numa aplicacao de frontend;
- b. Esta a ser validado um token de autenticao de um utilizador numa aplicacao de backend;
- c. A funcao verifyToken nao funcionara e resultara num erro 500 sempre que executada;
- d. A funcao verifyToken atua como funcao de middleware na aplicacao web.

P7. Tendo em consideracao a figura (@NgModule com AppComponent, PeopleComponent), indique as afirmacoes verdadeiras:

- a. Estamos perante uma aplicacao desenvolvida na framework ExpressJS;
- b. Estamos perante uma aplicacao desenvolvida na framework Angular;
- c. A aplicacao contem apenas o componente criado por defeito e outro criado pelo utilizador;
- d. Esta aplicacao esta a ser desenvolvida para o backend de uma aplicacao web.

Parte 2 - Verdadeiro ou Falso (3 valores)

V/F 1. "Uma pagina da internet so pode enviar dados para um servidor backend utilizando o elemento form em HTML com o metodo POST ou GET."

V/F 2. "JavaScript e uma linguagem de programacao que pode ser utilizada no frontend e backend."

V/F 3. "JavaScript e a linguagem de programacao utilizada em aplicacoes escritas com a framework Angular."

V/F 4. "Node.js e uma framework para escrita de aplicacoes no backend."

V/F 5. "O padrao de software MVC nao pode ser utilizado em aplicacoes escritas com a framework Angular."

V/F 6. "CSS e uma linguagem utilizada para formatar o aspeto de paginas da internet."

Parte 3 - Resposta Aberta (10 valores)

Aberta 1. Indique o que entende pelo conceito cliente-servidor em aplicacoes web. (1.5 val)

Aberta 2. Indique as diferentes formas possiveis para incluir CSS numa pagina de internet escrita em HTML. (1.5 val)

Aberta 3. Observe a figura (routing Angular com canActivate: [AuthGuardService]). Descreva o tipo de aplicacao, objetivo, funcionalidades e correcoes necessarias. (2 val)

Aberta 4. Observe a figura (pagina de multiplicacao com erros). Descreva a funcionalidade e os erros. (2 val)

Aberta 5. Onde e de que forma devemos validar a informacao submetida por utilizadores em aplicacoes web. (3 val)

EXAME 2021/2022 - Epoca Normal

Duracao: 1h40min | Parte 1: 8 valores | Parte 2: 2 valores | Parte 3: 10 valores

Parte 1 - Escolha Multipla (8 valores)

P1. Indique as afirmacoes verdadeiras:

- a. HTML e uma linguagem de programacao para a web;
- b. CSS permite estruturar o conteudo de pagina web;
- c. JavaScript apenas permite formatar os elementos de pagina web;
- d. E possivel submeter um formulario numa pagina HTML sem utilizar CSS ou JavaScript.

P2. Indique as afirmacoes verdadeiras sobre template engines:

- a. Angular possui um template engine que gera as paginas diretamente no browser;
- b. Podemos utilizar o template engine mongoose para gerar paginas HTML dinamicamente no servidor;
- c. O template engine EJS, num projeto gerado atraves do express generator, guarda os templates dentro da pasta views do projeto;
- d. Para utilizar template engines numa aplicacao web, necessitamos de utilizar, obrigatoriamente, APIs REST e operacoes CRUD.

P3. Atendendo as caracteristicas da framework ExpressJS, identifique as afirmacoes verdadeiras:

- a. ExpressJS e uma framework para desenvolver aplicacoes que correm exclusivamente no browser;
- b. O package manager npm pode ser utilizado para gerir as dependencias da nossa aplicacao;
- c. O ficheiro package.json guarda informacao sobre as dependencias da nossa aplicacao e comandos para serem executados com npm;
- d. ExpressJS e uma framework que cria aplicacoes que devem correr com um runtime de JavaScript como o NodeJS.

P4. Da seguinte lista selecione as afirmacoes verdadeiras sobre a framework Angular:

- a. Angular e uma framework para desenvolvimento de aplicacoes no backend;
- b. Angular apenas permite o uso de um componente por pagina/rota;
- c. Em angular nao e possivel utilizar os formularios de HTML com o elemento `<form></form>`;
- d. A linguagem de programacao de uma aplicacao em Angular e o TypeScript.

P5. Indique as afirmacoes verdadeiras sobre o desenvolvimento de servicos REST:

- a. Em servicos REST a informacao e trocada em formato binario entre cliente e servidor;
- b. Servicos REST nao permitem o uso do formato XML para troca de informacao;
- c. E possivel utilizar os metodos HTTP como GET, POST, PUT, DELETE para mapear operacoes CRUD;
- d. O modulo swagger-ui permite documentar e criar um ambiente de testes para APIs REST na framework ExpressJS.

P6. Observe o excerto de codigo (template EJS com `forEach`). Indique as afirmacoes verdadeiras:

- a. O código é executado por um template engine no servidor gerando um ficheiro HTML enviado para o cliente;
- b. O código é enviado para o cliente e depois executado para mostrar a informação no browser;
- c. Se a variável `newsList` estiver vazia nenhuma informação é mostrada no browser do cliente;
- d. Independentemente do número de news, apenas a primeira é mostrada no browser do cliente.

P7. Tendo em consideração o excerto de código (`mongoose.Schema`). Indique as afirmações verdadeiras:

- a. Estamos perante uma aplicação desenvolvida na framework Angular;
- b. Estamos perante uma aplicação desenvolvida na framework ExpressJS;
- c. O excerto de código cria um modelo de dados e um objeto para interagir com uma coleção da base de dados;
- d. O excerto de código representa a criação de uma classe de modelo de dados em TypeScript.

P8. Tendo em consideração o excerto de código (`JWTInterceptorService implements HttpInterceptor`). Indique as afirmações verdadeiras:

- a. O excerto de código é executado sempre que existe um pedido HTTP na aplicação;
- b. O excerto de código cria um token JWT e envia para o cliente;
- c. O excerto de código cria um header com o nome `x-access-token` num pedido HTTP;
- d. O excerto de código gere o acesso de um utilizador às rotas de uma página web.

Parte 2 - Verdadeiro ou Falso (2 valores)

V/F 1. "O objeto `XMLHttpRequest` em JavaScript permite obter informação de um servidor sem o uso de formulários e sem a necessidade de fazer reload a uma página da internet."

V/F 2. "Um formulário para login de utilizadores pode utilizar o método GET para enviar informação de autenticação para o servidor."

V/F 3. "As aplicações Angular executam no runtime NodeJS."

V/F 4. "Um serviço REST com o método PUT não deve permitir o envio de informação de um cliente para o servidor."

Parte 3 - Resposta Aberta (10 valores)

Aberta 1. Descreva o significado dos conceitos backend e frontend no desenvolvimento de uma aplicação web. (0.5 val)

Aberta 2. No desenvolvimento web indique o que entende pelo termo MEAN stack. (0.5 val)

Aberta 3. As operacoes CRUD estao relacionadas com o desenvolvimento de varios componentes web. Indique de que se tratam estas operacoes e 2 casos de uso. (1 val)

Aberta 4. Considere o padrao de software MVC. Descreva em que consiste este padrao e como o pode aplicar numa aplicacao que utiliza ExpressJS. (1.5 val)

Aberta 5. Considere a seguranca de aplicacoes web. Indique como podemos adicionar autenticao e autorizacao a uma aplicacao web que use frameworks para o frontend e backend. (3 val)

Aberta 6. Observe a figura (backend processa `app.post("/result")` com username e password). Escreva o conteudo do ficheiro `demo.html` de forma a ficar funcional. (1.5 val)

EXAME 2024/2025 - Epoca Normal

Duracao: 1h30min | Parte 1: 8 valores | Parte 2: 2 valores | Parte 3: 10 valores

Parte 1 - Escolha Multipla (8 valores)

Cada pergunta vale 1 valor. Indicar todas as opcoes corretas. Opcao incorreta penaliza 0.5 valores.

P1. Considere o uso da ferramenta NPM nos projetos web desenvolvidos durante o semestre. Indique todas as afirmacoes que indicam funcionalidades desta ferramenta:

- a. Instalar e configurar uma base de dados MongoDB;
- b. Iniciar projetos web com o comando `npm start`;
- c. Permite gerir dependencias de projetos web;
- d. Permite criar componentes em aplicacoes escritas com a framework Angular.

P2. Indique quais dos seguintes comandos podem ser executados numa aplicacao da framework Angular:

- a. `ng generate component my-page`;
- b. `npm start`;
- c. `npm install mongoose --save`;
- d. `npm new my-app`

- P3.** Indique as afirmacoes verdadeiras sobre os componentes do tipo servico em Angular:
- a. Permitem ser injetados em multiplos componentes Angular;
 - b. Podemos manter o estado global da aplicacao em variaveis internas de um servico e partilha-las com outros varios componentes;
 - c. Este componente tem de ter sempre uma interface escrita em HTML;
 - d. Podem ser utilizados em conjunto com o padrao de software observable para manter todos os componentes visuais atualizados.
- P4.** Considerando a framework ExpressJS utilizada em Node.js, indique as afirmacoes verdadeiras:
- a. Podemos criar componentes com a linguagem de programacao JavaScript e padrao de software MVC;
 - b. Podemos utilizar template engines para gerar paginas HTML dinamicamente no servidor;
 - c. Podemos aceder diretamente a uma base de dados para enviar e guardar informacao com o modulo mongoose;
 - d. Podemos executar funcoes JavaScript da framework ExpressJS diretamente a partir de um browser de internet.
- P5.** Indique todas as afirmacoes verdadeiras:
- a. Angular e uma framework de desenvolvimento fullstack;
 - b. HTML e uma linguagem criada para formatar conteudos em paginas web;
 - c. Node.js e a linguagem de programacao mais utilizada no desenvolvimento de aplicacoes Web;
 - d. Angular permite a reutilizacao de componentes no desenvolvimento de aplicacoes web.
- P6.** Considere o excerto HTML com 3 elementos usando id="mvp" e getElementById("mvp"). Indique as afirmacoes verdadeiras:
- a. Ao clicar no botao Executa a pagina nao e alterada;
 - b. Ao clicar no botao Executa o texto "Change Me!" altera a sua cor para vermelho;
 - c. A funcao document.getElementById("mvp") retorna todos os elementos da pagina com o id igual ao valor "mvp";
 - d. Ao clicar no botao Executa todo o texto da pagina fica com a cor vermelho.
- P7.** Considere o excerto de codigo de um middleware Express com JWT (jwt.verify + verificacao de decoded.role). Indique as afirmacoes verdadeiras:
- a. Esta a ser usada a framework Angular;
 - b. A funcao usa o modulo JWT para verificacao de um token para efeitos de autenticao e autorizacao;
 - c. A funcao verify e a ultima a ser executada numa rota da aplicacao ExpressJS;
 - d. Por estarmos a utilizar autenticao com tokens, estamos obrigatoriamente a utilizar uma API REST.
- P8.** Observe o excerto de codigo (@Injectable, HttpClient, getItem(): Observable<Item>, addItem()). Indique as afirmacoes verdadeiras:
- a. Esta demonstrado o consumo de uma API REST por uma aplicacao escrita em ExpressJS;
 - b. Esta demonstrado o consumo de uma API REST por uma aplicacao escrita em Angular;
 - c. A funcao getItem retorna imediatamente com o valor do Item guardado no servidor bloqueando ate estar completa;
 - d. O codigo permite aferir com certeza absoluta que esta a ser utilizada uma base de dados no backend.

Parte 2 - Verdadeiro ou Falso (2 valores)

V/F 1. "A validacao de dados de input deve ser apenas realizada no frontend de uma aplicacao web para garantir que so dados corretos chegam ao backend."

V/F 2. "CSS é considerada uma linguagem de programação pois permite formatar e realizar animações em páginas HTML."

V/F 3. "O módulo JWT não é adequado para garantir autenticação e autorização em páginas web codificadas com a framework ExpressJS e o template engine EJS."

V/F 4. "É possível utilizar a base de dados MongoDB no frontoffice de uma aplicação web para guardar dados da aplicação de forma persistente."

Parte 3 - Resposta Aberta (10 valores)

Aberta 1. Indique o que entende pelo conceito cliente-servidor em aplicações web. (1 val)

Aberta 2. Explique o que são serviços REST no âmbito do desenvolvimento de aplicações para a web. (2 val)

Aberta 3.1. Identifique e corrija os erros presentes no código do Click Counter. (1 val)

Aberta 3.2. Como podemos melhorar a página para que a contagem de cliques continue e não seja reiniciada ao reabrir a página? (1 val)

Aberta 4. Comente a frase: "Os componentes criados na framework Angular cumprem o padrão MVC". (1.5 val)

Aberta 5. Quais as funções dos componentes Guard e Intercept numa aplicação escrita com a framework Angular? (1.5 val)

SOLUCOES COMPLETAS

Exercicios de Treino PAW - Por Fases

Consulta as solucoes apenas depois de teres tentado resolver os exercicios por conta propria. A pratica ativa e a melhor forma de preparacao!

FASE 1 - Fundamentos (HTML, CSS, Web)

Escolha Multipla

1.1 No modelo cliente-servidor, quem inicia a comunicacao?

Resposta:

c) O cliente (browser) inicia a comunicacao enviando pedidos HTTP ao servidor.

1.2 Qual destes NAO e um metodo HTTP valido?

Resposta:

c) CREATE nao e um metodo HTTP. E uma operacao CRUD. O metodo HTTP equivalente e POST.

1.3 Qual o codigo de status HTTP correto para "utilizador nao autenticado"?

Resposta:

b) 401 Unauthorized = nao autenticado (falta login). 403 Forbidden = autenticado mas sem permissao.

1.4 Qual das seguintes afirmacoes e verdadeira?

Resposta:

b) CSS e uma linguagem de estilos (apresentacao). HTML e de marcacao, Node.js e um runtime, MongoDB e NoSQL.

1.5 Qual e o metodo HTTP idempotente usado para atualizar dados?

Resposta:

b) PUT e idempotente e e o metodo HTTP equivalente a operacao CRUD "Update". POST nao e idempotente. CREATE nao e um metodo HTTP.

1.6 (EXTRA) O que se encontra dentro da tag <head> de um documento HTML?

Resposta:

b) O <head> contem metadados. O conteudo do <head> NAO e visivel na pagina.

1.7 (EXTRA) Qual destes elementos HTML e inline?

Resposta:

c) e inline - nao inicia nova linha, ocupa so o espaco necessario. Os restantes sao block-level.

Verdadeiro ou Falso

1.8 "HTML e uma linguagem de programacao porque permite criar paginas web."

Resposta:

FALSO. HTML é uma linguagem de marcação (HyperText Markup Language). Define a estrutura/conteúdo, mas não tem lógica de programação (loops, condições, etc.).

1.9 "Um formulário HTML pode submeter dados ao servidor sem JavaScript."

Resposta:

VERDADEIRO. Basta usar `<form action="/rota" method="POST">` com um botão `<button type="submit">`. O browser submete nativamente.

1.10 "É seguro usar o método GET para enviar passwords num formulário de login."

Resposta:

FALSO. NUNCA usar GET para dados sensíveis. O GET expõe os dados na query string do URL. Usar sempre POST para login.

1.11 "O código HTTP 403 Forbidden indica que o utilizador não está autenticado."

Resposta:

FALSO. 403 Forbidden = autenticado mas sem permissão (autorização). 401 Unauthorized = não autenticado (falta login). São diferentes!

1.12 (EXTRA) "A tag `<img>` requer uma tag de fecho `</img>` para ser válida."

Resposta:

FALSO. `<img>` é um elemento self-closing (vazio). A sintaxe correta é `` sem tag de fecho.

1.13 (EXTRA) "No Box Model do CSS, o padding está entre o content e o border."

Resposta:

VERDADEIRO. A ordem de dentro para fora é: Content -> Padding -> Border -> Margin.

Resposta Aberta

1.14 Explica o modelo cliente-servidor na web, incluindo os papéis de cada parte e o fluxo típico de comunicação. (1-1.5 val)

Resposta:

Cliente (browser): Executa no computador do utilizador. Responsável pela interface (frontend). Inicia a comunicação enviando pedidos HTTP. Não deve ter acesso direto a bases de dados.

Servidor (ex: Node.js + Express): Aplicação num computador remoto. Recebe pedidos HTTP, processa-os e devolve respostas. Accede a bases de dados e recursos protegidos.

Fluxo: 1. Utilizador escreve URL -> browser envia pedido HTTP ao servidor. 2. Servidor processa (middleware -> controller -> model/BD). 3. Servidor devolve resposta HTTP (HTML, JSON, código de status). 4. Browser renderiza a resposta.

1.15 Indica as 3 formas de adicionar CSS a uma página HTML. Qual é a recomendada e porque? (1-1.5 val)

Resposta:

1. Inline - Diretamente no elemento via atributo style: `<p style="color: red;">Texto</p>`

2. Internal - Dentro de uma tag `<style>` no `<head>` do documento HTML.

3. External - Num ficheiro .css externo, referenciado com `<link rel="stylesheet" href="ficheiro.css">` no `<head>`. (RECOMENDADA)

Motivo: Separa apresentação da estrutura. Reutilizável em múltiplas páginas. Browser pode fazer cache. Facilita manutenção.

FASE 2 - JavaScript & DOM

Escolha Multipla

2.1 Qual e a diferenca entre `==` e `===` em JavaScript?

Resposta:

c) `===` (strict equality) compara valor E tipo. `==` (loose equality) compara apenas o valor, fazendo coercao de tipos. Boa pratica: usar sempre `===`.

2.2 O que retorna `document.getElementById("xyz")`?

Resposta:

c) `getElementById` retorna UM UNICO elemento - o primeiro com esse ID. IDs devem ser unicos no HTML.

2.3 Qual e a diferenca entre `localStorage` e `sessionStorage`?

Resposta:

b) `localStorage` sobrevive ao fechar do browser (permanente). `sessionStorage` e eliminado quando a tab/sessao e fechada.

2.4 Qual a diferenca entre `var`, `let` e `const`?

Resposta:

b) `var` tem scope de funcao e sofre hoisting. `let` e `const` (ES6) tem scope de bloco. `const` nao pode ser reatribuido.

2.5 O que faz o AJAX?

Resposta:

b) AJAX permite fazer pedidos HTTP assincronos ao servidor e atualizar partes da pagina sem a recarregar.

2.6 (EXTRA) Qual dos seguintes estados NAO e um estado valido de uma Promise?

Resposta:

c) Os estados de uma Promise sao: `pending`, `fulfilled` ou `rejected`. `"loading"` nao existe.

2.7 (EXTRA) Qual e a diferenca entre `innerHTML` e `textContent`?

Resposta:

b) `innerHTML` interpreta o conteudo como HTML (renderiza tags). `textContent` trata tudo como texto puro.

Verdadeiro ou Falso

2.8 "Em JavaScript, `var` sofre hoisting mas `let` e `const` nao."

Resposta:

VERDADEIRO. A declaracao `var` "sobe" ao topo da funcao (hoisting). `let` e `const` (ES6) nao sofrem hoisting - sao block-scoped.

2.9 "AJAX requer sempre que a pagina inteira seja recarregada para funcionar."

Resposta:

FALSO. AJAX permite atualizar partes da pagina sem recarregar toda a pagina. E exatamente essa a sua finalidade.

2.10 "JSON.parse() converte um objeto JavaScript numa string JSON."

Resposta:

FALSO. JSON.parse() converte uma string JSON -> objeto JavaScript. O inverso (JSON.stringify()) converte objeto -> string JSON.

2.11 (EXTRA) "Arrow functions e function declarations comportam-se de forma identica em todos os contextos."

Resposta:

FALSO. Arrow functions nao tem o seu proprio this, nao sofrem hoisting, e nao podem ser usadas como construtores (new).

2.12 (EXTRA) "O readyState == 4 numa chamada XMLHttpRequest indica que a resposta esta completa."

Resposta:

VERDADEIRO. readyState == 4 significa que a operacao esta completa. Deve-se verificar tambem status == 200 para garantir que foi bem-sucedida.

Resposta Aberta

2.13 O seguinte codigo HTML/JS tem 3 erros. Identifica e corrige cada um. (1-2 val)

```
<html>
<body>
  <h1>Click Counter</h1>
  <p>Click Number: <span id="contador">0</span></p>
  <button>Click Me</button>
  <script>
    function contarClique() {
      numeroCliquess++;
      document.getElementById(counter).textContent = numeroCliquess;
    }
  </script>
</body>
</html>
```

Resposta:

Erro 1: A variavel numeroCliquess e usada sem ser declarada -> var numeroCliquess = 0;

Erro 2: getElementById(counter) - counter e variavel undefined -> Deve ser string: getElementById("contador")

Erro 3: O botao nao tem nenhum evento associado -> Adicionar onclick="contarClique()" ao botao.

Codigo corrigido:

```
<button onclick="contarClique()">Click Me</button>
<script>
  var numeroCliquess = 0;
  function contarClique() {
    numeroCliquess++;
    document.getElementById("contador").textContent = numeroCliquess;
  }
</script>
```

2.14 Explica como usar localStorage para manter dados entre recarregamentos de pagina. (1 val)

Resposta:

O localStorage e uma API do browser que permite guardar dados no formato chave-valor de forma permanente (sobrevive ao fechar o browser). Os dados sao strings, pelo que para objetos/arrays e preciso usar JSON.stringify() e JSON.parse().

```
// 1. Ao carregar a pagina, ler dados guardados
var dados = JSON.parse(localStorage.getItem('meusDados')) || valorPadrao;
// 2. Mostrar dados no HTML
document.getElementById('elemento').textContent = dados;
// 3. Ao alterar dados, guardar no localStorage
function atualizar(novoDado) {
```

```
dados = novoDado;
document.getElementById("elemento").textContent = dados;
localStorage.setItem("meusDados", JSON.stringify(dados));
}
```

FASE 3 - Backend: Node.js, Express, BD & REST

Escolha Multipla

3.1 O que e o Node.js?

Resposta:

c) Node.js e um runtime JavaScript baseado no motor V8 do Chrome. Nao e uma linguagem, nem um framework, nem um servidor web por si so.

3.2 Qual a diferenca entre req.params, req.query e req.body?

Resposta:

b) req.params -> parametros de rota (ex: /produto/:id). req.query -> query strings (ex: ?q=teste). req.body -> corpo do pedido (POST/PUT).

3.3 O que o NPM NAO faz?

Resposta:

c) Criar componentes Angular e feito pelo Angular CLI (ng generate component), nao pelo NPM.

3.4 O que e middleware no Express?

Resposta:

b) Middleware sao funcoes que se executam entre o pedido HTTP e a resposta final. E fundamental chamar next() para passar ao proximo middleware.

3.5 Qual tag EJS produz output HTML escaped?

Resposta:

b) <%= %> produz output com HTML escaping (seguro contra XSS). <% %> executa codigo sem output. <%- %> produz output sem escaping.

3.6 (EXTRA) No padrao MVC aplicado a Express, onde ficam os schemas/models Mongoose?

Resposta:

c) No MVC, os schemas/models Mongoose ficam na pasta models/. Views ficam em views/ (templates EJS), controllers em controllers/.

3.7 (EXTRA) O que distingue uma BD relacional (SQL) de uma BD NoSQL como o MongoDB?

Resposta:

b) SQL tem schema rigido, transacoes ACID e escalabilidade vertical. NoSQL (MongoDB) tem schema flexivel, consistencia eventual e escalabilidade horizontal.

3.8 (EXTRA) Qual e o middleware de erros correto no Express?

Resposta:

b) O middleware de erros no Express tem obrigatoriamente 4 parametros: (err, req, res, next).

Verdadeiro ou Falso

3.9 *"Express e um framework que executa no browser do utilizador."*

Resposta:

FALSO. Express e um framework web server-side que corre em Node.js no servidor. Nao executa no browser.

3.10 *"REST APIs comunicam APENAS em JSON."*

Resposta:

FALSO. REST permite JSON e XML (e outros formatos de texto). Nao esta limitado a JSON. REST NAO usa formato binario.

3.11 *"O Mongoose e uma base de dados NoSQL."*

Resposta:

FALSO. Mongoose e um driver/ODM (Object Data Modeling) para MongoDB em Node.js. A base de dados e o MongoDB.

3.12 *"JWT (JSON Web Token) obriga a usar uma REST API."*

Resposta:

FALSO. JWT pode ser usado com qualquer tipo de aplicacao web. Nao depende do tipo de rendering nem da arquitetura.

3.13 *"A ordem do middleware no Express e irrelevante."*

Resposta:

FALSO. A ordem importa! O middleware e executado sequencialmente. Um middleware de autenticao depois de uma rota nao protege essa rota.

3.14 (EXTRA) *"O NPM pode ser usado para instalar o MongoDB no sistema."*

Resposta:

FALSO. NPM instala o driver de MongoDB (ex: mongoose), nao o proprio MongoDB. A instalacao do MongoDB e feita separadamente.

3.15 (EXTRA) *"A leitura bloqueante (sincrona) com fs.readFileSync() e a abordagem recomendada em Node.js."*

Resposta:

FALSO. Em Node.js, a abordagem recomendada e nao bloqueante (assincrona) com fs.readFile() e callbacks/promises.

Resposta Aberta

3.16 *Explica o padrao MVC e como se aplica numa aplicacao Express com MongoDB. (1.5 val)*

Resposta:

Model: Responsavel pelos dados e logica de negocio. Em Express com MongoDB, corresponde aos schemas e models Mongoose na pasta models/.

View: Responsavel pela apresentacao/interface. Em Express, sao os templates EJS na pasta views/.

Controller: Liga o Model a View. Processa os pedidos HTTP, obtem dados do Model e envia-os para a View (pasta controllers/).

```
// Model
const Produto = mongoose.model('Produto', new mongoose.Schema({ nome: String, preco: Number }));

// Controller
exports.listarProdutos = async (req, res) => {
```

```
const produtos = await Produto.find();
res.render('produtos', { produtos });
};
```

3.17 Explica a diferenca entre autenticacao e autorizacao. Indica os codigos HTTP associados e da exemplos praticos. (1.5 val)

Resposta:

	Autenticacao	Autorizacao
Pergunta	"Quem es tu?"	"O que podes fazer?"
Quando	No login	Apos login, em cada pedido
Codigo HTTP	401 Unauthorized	403 Forbidden

401 Unauthorized = nao autenticado (falta login). 403 Forbidden = autenticado mas sem permissao.

Autenticacao: Verificar a identidade do utilizador. Ex: login com email + password. O servidor gera JWT com `jwt.sign()`.

Autorizacao: Verificar se o utilizador autenticado tem permissao. Ex: utilizador normal tenta aceder a `/admin`. O role nao e "ADMIN".

3.18 Descreve as propriedades de uma REST API e como implementarias uma operacao CRUD completa em Express. (2 val)

Resposta:

Propriedades REST: Stateless, Cacheable, Interface uniforme, Sistema em camadas, Dados em texto (JSON ou XML), nunca em binario.

```
// CREATE -> POST
app.post('/api/produtos', async (req, res) => { const p = new Produto(req.body); await p.save();
res.status(201).json(p); });

// READ -> GET
app.get('/api/produtos', async (req, res) => { const produtos = await Produto.find();
res.json(produtos); });

// UPDATE -> PUT
app.put('/api/produtos/:id', async (req, res) => { const p = await
Produto.findByIdAndUpdate(req.params.id, req.body, { new: true }); res.json(p); });

// DELETE -> DELETE
app.delete('/api/produtos/:id', async (req, res) => { await
Produto.findByIdAndDelete(req.params.id); res.status(204).send(); });
```

FASE 4 - Angular & Full-Stack

Escolha Multipla

4.1 Qual e a linguagem usada no Angular?

Resposta:

c) Angular usa TypeScript, um superset de JavaScript criado pela Microsoft que adiciona tipos estaticos e compilacao.

4.2 Qual o comando para criar um novo projeto Angular?

Resposta:

b) `ng new my-app` e o comando correto do Angular CLI. "npm new" NAO EXISTE! E uma armadilha frequente de exame.

4.3 Em Angular, qual sintaxe implementa two-way data binding?

Resposta:

d) [(ngModel)]="variavel" implementa two-way binding (bidirecional). Requer FormsModule.

4.4 O que é @Injectable() em Angular?

Resposta:

b) @Injectable() é um decorador que marca uma classe Angular como injetável via Dependency Injection.

4.5 Qual a diferença entre Guard e Interceptor em Angular?

Resposta:

b) Guard implementa CanActivate e protege rotas/páginas. Interceptor implementa HttpInterceptor e modifica pedidos HTTP.

4.6 Qual hook do ciclo de vida Angular é o mais usado e executado uma única vez após inicialização?

Resposta:

b) ngOnInit() é executado uma única vez após a inicialização do componente. É o mais usado para carregar dados iniciais.

4.7 (EXTRA) O que é uma SPA (Single Page Application)?

Resposta:

b) SPA é uma aplicação web com uma única página HTML que carrega conteúdo dinamicamente via JavaScript.

4.8 (EXTRA) Como é que um componente filho recebe dados do componente pai em Angular?

Resposta:

b) O decorador @Input() no componente filho permite receber dados do pai via property binding.

Verdadeiro ou Falso

4.9 "Angular é um framework fullstack que executa tanto no frontend como no backend."

Resposta:

FALSO. Angular é um framework client-side (frontend). Executa no browser, NÃO no runtime Node.js nem no servidor.

4.10 "Em Angular, a rota de login deve ter canActivate: [AuthGuard] para ser segura."

Resposta:

FALSO. Rotas de login e register NÃO devem ter canActivate! Isso impediria utilizadores não-autenticados de fazerem login.

4.11 "TypeScript é um superset de JavaScript - todo código JS válido é também TS válido."

Resposta:

VERDADEIRO. TypeScript é um superset de JavaScript. Mantém compatibilidade total com JS mas adiciona tipos estáticos e compilação.

4.12 "Serviços Angular podem ser injetados em múltiplos componentes."

Resposta:

VERDADEIRO. Serviços Angular usam Dependency Injection (DI) com @Injectable({ providedIn: 'root' }), criando uma instância singleton partilhada.

4.13 (EXTRA) "O ng build gera o build de producao na pasta node_modules/."

Resposta:

FALSO. ng build gera o build de producao na pasta dist/, nao em node_modules/.

4.14 (EXTRA) "Um Observable em Angular retorna o valor imediatamente de forma sincrona."

Resposta:

FALSO. Observables sao assincronos e nao bloqueantes. O valor so e obtido quando se chama .subscribe().

Resposta Aberta

4.15 Explica a diferenca entre Guards e Interceptors em Angular. Da exemplos de codigo de cada um. (1.5 val)

Resposta:

	Guard	Interceptor
Protege	Rotas/paginas	Pedidos HTTP
Interface	CanActivate	HttpInterceptor
Quando age	Antes de navegar para uma rota	Antes de enviar um pedido HTTP
Exemplo	Bloquear acesso a /admin	Adicionar token JWT ao header

Guard implementa CanActivate e protege rotas/paginas - verifica antes de navegar.

Interceptor implementa HttpInterceptor e modifica pedidos HTTP antes de serem enviados.

```
@Injectable()
export class AuthGuard implements CanActivate {
  canActivate(): boolean {
    if (localStorage.getItem('token')) return true;
    this.router.navigate(['/login']);
    return false;
  }
}

@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const token = localStorage.getItem('token');
    const cloned = req.clone({
      headers: req.headers.set('Authorization', 'Bearer ' + token)
    });
    return next.handle(cloned);
  }
}
```

4.16 Como funciona um servico Angular? Explica o que sao @Injectable(), HttpClient e Observable. (1.5 val)

Resposta:

@Injectable({ providedIn: "root" }): Decorador que marca a classe como injetavel via DI. Cria instancia singleton.

HttpClient: Modulo Angular para fazer pedidos HTTP a APIs REST. Retorna Observables.

Observable (RxJS): Representa um fluxo de dados assincrono. O componente usa .subscribe() para obter os dados.

```
@Injectable({ providedIn: 'root' })
export class ProductService {
  constructor(private http: HttpClient) {}
  getProducts(): Observable<Product[]> {
    return this.http.get<Product[]>('http://api/products');
  }
}
```

```
// No componente:
ngOnInit() {
  this.productService.getProducts().subscribe(data => { this.products = data; });
}
```

FASE 5 - Seguranca & Identificacao de Tecnologia

Escolha Multipla

5.1 *Porque e que a validacao deve ser feita no frontend E no backend?*

Resposta:

b) Frontend: melhor UX. Backend: seguranca - o utilizador pode desativar JS, enviar pedidos com Postman/curl, ou manipular o HTML nos DevTools.

5.2 *Qual excerto de codigo identifica um servico Angular?*

Resposta:

b) @Injectable(), Observable e HttpClient sao indicadores de um servico Angular. a) e Express middleware, c) e Mongoose, d) e JavaScript DOM.

5.3 *Qual das seguintes afirmacoes sobre seguranca e FALSA?*

Resposta:

c) Validacao apenas no frontend NAO e suficiente. A validacao DEVE ser feita em AMBOS os lados.

5.4 (EXTRA) *Que tecnologia usarias se visses <%= %> e <% %> num ficheiro?*

Resposta:

c) <%= %> e <% %> sao tags EJS (Embedded JavaScript). <%= %> produz output HTML escaped, <% %> executa codigo sem output.

5.5 (EXTRA) *Qual o principal risco de usar innerHTML em vez de textContent?*

Resposta:

b) innerHTML interpreta HTML, o que pode levar a ataques XSS (Cross-Site Scripting) se o conteudo vier de input do utilizador.

Verdadeiro ou Falso

5.6 *"MongoDB pode correr diretamente no frontend (browser)."*

Resposta:

FALSO. Bases de dados correm no servidor, nao no browser. O frontend comunica com a BD atraves do backend (Express/Node.js).

5.7 *"JavaScript funciona apenas no frontend (browser)."*

Resposta:

FALSO. JavaScript funciona no frontend (browser) E no backend (Node.js). Node.js e um runtime que permite executar JS no servidor.

5.8 *"Um servico Angular pode ter um template HTML associado."*

Resposta:

FALSO. Servicos Angular NAO tem template HTML. Sao classes TypeScript puras com @Injectable(). Apenas componentes tem templates.

5.9 (EXTRA) "CORS (Cross-Origin Resource Sharing) e uma restricao imposta pelo servidor."

Resposta:

FALSO. CORS e imposto pelo browser (Same-Origin Policy). A solucao e configurar o servidor para enviar headers CORS (app.use(cors())).

5.10 (EXTRA) "O Swagger/OpenAPI serve para executar APIs REST automaticamente."

Resposta:

FALSO. Swagger/OpenAPI e um padrao para documentar APIs REST. Nao executa as APIs automaticamente.

Resposta Aberta

5.11 *Explica porque e que a validacao de dados deve ser feita no frontend E no backend. Da 3 razoes concretas para validar no backend. (2-3 val)*

Resposta:

A validacao DEVE ser feita em AMBOS os lados:

Frontend: Melhora a UX com feedback imediato. Usa atributos HTML (required, minlength) ou validacao Angular.

Backend: Garante seguranca. O frontend pode ser contornado!

3 razoes para validar no backend:

1. JavaScript desativado: O utilizador pode desativar JS no browser, eliminando toda a validacao frontend.
 2. Pedidos diretos: O utilizador pode enviar pedidos diretamente com Postman ou curl, contornando a interface web.
 3. Manipulacao do codigo: O utilizador pode abrir os DevTools e modificar o HTML ou JS antes de submeter.
- Conclusao: Confiar apenas no frontend e uma falha de seguranca grave. O backend e a ultima linha de defesa.

5.12 *Dado o seguinte excerto de codigo (AuthInterceptor), identifica a tecnologia usada e explica o que o codigo faz: (1.5 val)*

```
@Injectable({ providedIn: 'root' })
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const token = localStorage.getItem('token');
    const cloned = req.clone({
      headers: req.headers.set('Authorization', 'Bearer ' + token)
    });
    return next.handle(cloned);
  }
}
```

Resposta:

Tecnologia: Angular Interceptor (interceptador HTTP).

Identificacao: Presenca de @Injectable(), HttpInterceptor, intercept, HttpRequest, HttpHandler.

O que faz: 1. E um servico injetavel que implementa HttpInterceptor. 2. O metodo intercept() e chamado automaticamente antes de cada pedido HTTP. 3. Obtem o token JWT do localStorage. 4. Clona o pedido (req.clone()). 5. Adiciona o header Authorization: Bearer <token>. 6. Envia o pedido modificado com next.handle(cloned).

Proposito: Automatizar a inclusao do token JWT em todos os pedidos HTTP, centralizando a logica de autenticacao.

Exercicios Integrados - Solucoes

Escolha Multipla

I.1 Qual das seguintes afirmacoes sobre o express-generator e verdadeira?

Resposta:

b) O express-generator cria uma estrutura de projeto Express com a pasta views/ para templates (EJS por defeito), routes/, public/, etc.

I.2 Qual destas combinacoes de tecnologia esta errada?

Resposta:

c) Mongoose NAO e um template engine. E um driver/ODM para MongoDB. Templates engines sao EJS, Pug, Mustache, etc.

I.3 Em qual situacao usarias `res.render()` vs `res.json()`?

Resposta:

b) `res.render("view", data)` renderiza um template EJS e envia HTML. `res.json(obj)` envia dados JSON, tipicamente numa API REST consumida por um frontend Angular/React.

Verdadeiro ou Falso

I.4 "O Angular pode aceder diretamente ao MongoDB usando Mongoose no browser."

Resposta:

FALSO. Angular executa no browser e NUNCA deve aceder diretamente a BD. Mongoose e um modulo Node.js que corre no servidor.

I.5 "Node.js e single-threaded com I/O bloqueante."

Resposta:

FALSO. Node.js e single-threaded com I/O nao bloqueante (assincrono). E isso que permite lidar com muitas conexoes simultaneas.

I.6 "O npm start compila TypeScript e instala MongoDB."

Resposta:

FALSO. npm start executa o script "start" definido no package.json. Compilar TypeScript e tsc. Instalar MongoDB e feito separadamente.

Resposta Aberta

I.7 Uma aplicacao web tem um formulario de registo. Descreve todas as camadas de seguranca e validacao que implementarias, desde o frontend ate a base de dados. (3 val)

Resposta:

1. Frontend (Angular): Formulario com `method="POST"` (NUNCA GET para passwords). Validacao com `required`, `minlength`, `type="email"`, `type="password"`. Feedback visual imediato.
2. Comunicacao: Usar HTTPS em producao. Gerar JWT com `jwt.sign()` apos registo. Nunca enviar a password de volta ao frontend.
3. Backend (Express): Validar dados recebidos via middleware. Sanitizar inputs. Verificar se o email ja existe. Hash da password com `bcrypt`. Nunca guardar passwords em texto plano!
4. Base de dados (Mongoose): Schema com validacao (`required`, `unique`, `minlength`). `unique: true` no email previne duplicados.

Resumo: Validacao em 3 camadas (frontend -> backend -> BD) + HTTPS + password hashing + sanitizacao.

I.8 Corriga o seguinte código de um servidor Express que tem vários erros: (2 val)

```
const express = require('express');
const app = express();

app.get('/api/produto/:id', (req, res) => {
  const id = req.query.id;
  const produto = Produto.find(id);
  res.send(produto);
});

app.use(express.json());

app.post('/login', (req, res) => {
  const password = req.params.password;
  res.status(200);
});

app.listen(3000);
```

Resposta:

- Erro 1: req.query.id -> Deve ser req.params.id (o :id está na rota como parametro de rota).
- Erro 2: Produto.find(id) -> Deve ser await Produto.findById(id). Falta await e Produto não foi importado.
- Erro 3: express.json() está depois das rotas -> middleware deve ser registrado ANTES das rotas.
- Erro 4: req.params.password -> Num POST, os dados vêm no body: req.body.password.
- Erro 5: res.status(200) -> Falta enviar uma resposta: res.status(200).json({ msg: "OK" }).

```
const express = require('express');
const Produto = require('./models/Produto');
const app = express();
app.use(express.json()); // ANTES das rotas
app.get('/api/produto/:id', async (req, res) => {
  const produto = await Produto.findById(req.params.id);
  res.json(produto);
});
app.post('/login', (req, res) => {
  const password = req.body.password;
  res.status(200).json({ msg: 'Login OK' });
});
app.listen(3000);
```

SOLUCOES — EXAMES REAIS

EXAME 2020/2021

Parte 1 - Escolha Múltipla

P1. Indique as afirmações verdadeiras:

Resposta:

- Corretas: b, c, d
- a) FALSO - CSS é uma linguagem de estilos, não de programação.
 - b) VERDADEIRO - O <head> contém metadados, mas não o conteúdo visível.
 - c) VERDADEIRO - Podemos usar <form action="/rota" method="POST"> nativamente.
 - d) VERDADEIRO - Uma página pode conter apenas puro HTML.

P2. Entre as seguintes opções escolha os métodos HTTP que podem ser utilizados para enviar dados a um servidor numa API REST:

Resposta:

Corretas: a, b

a, b) GET e DELETE são métodos/verbos HTTP válidos.

c, d) CREATE e UPDATE são operações CRUD, NÃO métodos HTTP.

P3. Considerando a framework ExpressJS utilizada em Node.js, indique as afirmações verdadeiras:

Resposta:

Corretas: b, c

a) FALSO - A criação de "componentes com TypeScript" remete ao Angular. O ExpressJS usa JavaScript padrão.

b) VERDADEIRO - O Express suporta template engines como o EJS.

c) VERDADEIRO - O Mongoose e o módulo standard para ligar Node.js/Express ao MongoDB.

d) FALSO - ExpressJS corre no servidor, não no browser.

P4. Da seguinte lista selecione as afirmações verdadeiras sobre uma aplicação escrita com a framework Angular:

Resposta:

Corretas: c, d

a) FALSO - Nunca se liga o Angular diretamente a base de dados.

b) FALSO - Apps browser-side não têm acesso direto ao sistema de ficheiros.

c) VERDADEIRO - O Angular usa o package.json e o npm para gerir bibliotecas.

d) VERDADEIRO - O Angular é baseado numa arquitetura modular de componentes reutilizáveis.

P5. Indique as afirmações verdadeiras sobre o desenvolvimento de serviços REST:

Resposta:

Corretas: a, d

a) VERDADEIRO - O Swagger/OpenAPI gera documentação interativa e um ambiente de testes.

b) FALSO - Serviços REST comunicam em dados puros (tipicamente JSON).

c) FALSO - REST obriga a separação cliente-servidor.

d) VERDADEIRO - JWT permite autenticação e autorização via tokens.

P6. Observe a figura (`authController.verifyToken` com `jwt.verify`) e indique as afirmações verdadeiras:

Resposta:

Corretas: b, d

a) FALSO - É middleware Express (Backend), não Angular.

b) VERDADEIRO - É uma função de backend que valida tokens JWT.

c) FALSO - A função funcionaria se o token for válido.

d) VERDADEIRO - É um middleware (tem `next()`). Só quando a autenticação passa e que chama `next()`.

P7. Tendo em consideração a figura (`@NgModule` com `AppComponent`, `PeopleComponent`), indique as afirmações verdadeiras:

Resposta:

Corretas: b, c

a) FALSO - `@NgModule` é um decorador Angular, não Express.

b) VERDADEIRO - O decorador `@NgModule` em TypeScript indica `app.module.ts` do Angular.

c) VERDADEIRO - declarações contêm exatamente dois componentes.

d) FALSO - Angular é uma framework client-side (frontend).

Parte 2 - Verdadeiro ou Falso

V/F 1. "Uma pagina da internet so pode enviar dados para um servidor backend utilizando o elemento form em HTML com o metodo POST ou GET."

Resposta:

FALSO. O browser pode enviar dados de forma assincrona usando AJAX (via XMLHttpRequest) ou a Fetch API via JavaScript. Nao precisa de formularios.

V/F 2. "JavaScript e uma linguagem de programacao que pode ser utilizada no frontend e backend."

Resposta:

VERDADEIRO. JavaScript corre no browser (frontend) e em servidores (backend) gracias a runtimes como o Node.js.

V/F 3. "JavaScript e a linguagem de programacao utilizada em aplicacoes escritas com a framework Angular."

Resposta:

FALSO. A linguagem utilizada no Angular e o TypeScript (superset do JavaScript, com tipagem estatica e compilacao obrigatoria).

V/F 4. "Node.js e uma framework para escrita de aplicacoes no backend."

Resposta:

FALSO. Node.js nao e uma framework - e um runtime (ambiente de execucao). A framework para backend e o ExpressJS.

V/F 5. "O padrao de software MVC nao pode ser utilizado em aplicacoes escritas com a framework Angular."

Resposta:

FALSO. O Angular usa intrinsecamente o padrao MVC: Model = classes/servicos, View = template HTML, Controller = classe TypeScript do componente.

V/F 6. "CSS e uma linguagem utilizada para formatar o aspeto de paginas da internet."

Resposta:

VERDADEIRO. E a funcao exata do CSS - formatar e estilizar a apresentacao visual de paginas web.

Parte 3 - Resposta Aberta

Aberta 1. Indique o que entende pelo conceito cliente-servidor em aplicacoes web. (1.5 val)

Resposta:

Cliente (browser): Executa a interface grafica (frontend) e inicia a comunicacao enviando um pedido HTTP.
Servidor (ex: Node.js): Recebe o pedido HTTP, executa a logica de negocio, acede a bases de dados, e devolve uma resposta HTTP.
Fluxo: Cliente envia pedido HTTP -> Servidor processa -> Servidor devolve resposta HTTP -> Cliente renderiza.

Aberta 2. Indique as diferentes formas possiveis para incluir CSS numa pagina de internet escrita em HTML. (1.5 val)

Resposta:

1. Inline: Usando o atributo style diretamente na tag HTML (ex: <p style="color:red;">).
2. Internal: Colocando codigo CSS dentro de uma tag <style> na seccao <head>.
3. External: Referenciando um ficheiro externo com <link rel="stylesheet" href="style.css"> na seccao <head>. (RECOMENDADO)

Aberta 3. Observe a figura (routing Angular com canActivate: [AuthGuardService]). Descreva o tipo de aplicacao, objetivo, funcionalidades e correcoes necessarias. (2 val)

Resposta:

Tipo de Aplicacao: SPA (Single Page Application) desenvolvida em Angular.

Objetivo: Configurar o Routing (navegacao) da aplicacao, mapeando URLs para componentes Angular.

ERRO LOGICO: As rotas de login e register NAO devem ter canActivate! Isso impediria utilizadores nao-autenticados de fazerem login/registo. O Guard so deve proteger rotas como /profile.

Aberta 4. Observe a figura (pagina de multiplicacao com erros). Descreva a funcionalidade e os erros. (2 val)

Resposta:

Funcionalidade: A pagina pretende obter dois numeros a partir de inputs, multiplica-los ao clicar num botao e exibir o resultado.

Erro 1: O segundo input usa class="arg2" mas o JS usa getElementById("arg2"). Resolucao: Alterar class="arg2" para id="arg2".

Erro 2: O script usa getElementById("calcular") mas o botao nao tem ID. Resolucao: Adicionar id="calcular" a tag <button>.

Aberta 5. Onde e de que forma devemos validar a informacao submetida por utilizadores em aplicacoes web. (3 val)

Resposta:

A validacao DEVE ser feita em AMBOS os lados:

Frontend: Melhora a UX com feedback imediato. Usa validacoes HTML5 (required, minlength, type="email") ou validacao reativa do Angular.

Backend: Garante a Seguranca. A validacao frontend pode ser contornada por: desativar JS, enviar pedidos com Postman/curl, manipular o HTML/JS nos DevTools.

O servidor deve SEMPRE validar e sanitizar toda a informacao antes de interagir com a base de dados.

EXAME 2021/2022

Parte 1 - Escolha Multipla

P1. Indique as afirmacoes verdadeiras:

Resposta:

Corretas: d

a) FALSO - HTML e uma linguagem de marcacao, nao de programacao.

b) FALSO - CSS serve para apresentar/estilizar. O HTML e que estrutura o conteudo.

c) FALSO - JS adiciona logica e interatividade.

d) VERDADEIRO - Um formulario <form> HTML padrao submete dados sem CSS ou JS ativo.

P2. Indique as afirmacoes verdadeiras sobre template engines:

Resposta:

Corretas: a, c

a) VERDADEIRO - O Angular compila e renderiza os seus templates HTML diretamente no cliente.

b) FALSO - Mongoose e um driver de base de dados (ODM), NAO um template engine.

c) VERDADEIRO - No ExpressJS, o EJS guarda as views na pasta views/ por defeito.

d) FALSO - Template engines geram HTML no servidor sem precisar de APIs REST.

P3. Atendendo as caracteristicas da framework ExpressJS, identifique as afirmacoes verdadeiras:

Resposta:

Corretas: b, c, d

a) FALSO - ExpressJS corre no servidor via o runtime Node.js, NAO no browser.

- b) VERDADEIRO - O npm é usado para instalar e gerir dependências.
- c) VERDADEIRO - O package.json contém dependências, scripts e metadados.
- d) VERDADEIRO - Express é uma framework para Node.js.

P4. Da seguinte lista selecione as afirmações verdadeiras sobre a framework Angular:

Resposta:

Corretas: d

- a) FALSO - Angular é estritamente Frontend (client-side).
- b) FALSO - Permite compor páginas com dezenas de componentes reutilizáveis.
- c) FALSO - Angular suporta e incentiva o uso de <form>.
- d) VERDADEIRO - Todo o código Angular é escrito nativamente em TypeScript.

P5. Indique as afirmações verdadeiras sobre o desenvolvimento de serviços REST:

Resposta:

Corretas: c, d

- a) FALSO - REST troca informação em texto (JSON ou XML), NÃO em formato binário.
- b) FALSO - REST permite JSON e XML.
- c) VERDADEIRO - GET->Read, POST->Create, PUT->Update, DELETE->Delete.
- d) VERDADEIRO - O swagger-ui-express cria documentação visual e interativa.

P6. Observe o excerto de código (template EJS com forEach). Indique as afirmações verdadeiras:

Resposta:

Corretas: a, c

- a) VERDADEIRO - É um template EJS que corre no servidor. O servidor compila o HTML e envia HTML estático ao browser.
- b) FALSO - O código EJS NÃO é enviado para o cliente.
- c) VERDADEIRO - O bloco if previne a renderização se estiver vazia.
- d) FALSO - O bloco forEach percorre todos os elementos.

P7. Tendo em consideração o excerto de código (mongoose.Schema). Indique as afirmações verdadeiras:

Resposta:

Corretas: c

- a) FALSO - O código usa mongoose, que é um módulo de backend (Node.js).
- c) VERDADEIRO - Define um Schema e exporta um Model Mongoose para interagir com uma coleção MongoDB.
- d) FALSO - O código usa JavaScript (CommonJS com require), não TypeScript.

P8. Tendo em consideração o excerto de código (JWTInterceptorService implements HttpInterceptor). Indique as afirmações verdadeiras:

Resposta:

Corretas: a, c

- a) VERDADEIRO - Um Interceptor HTTP do Angular é executado automaticamente em todos os pedidos HTTP.
- b) FALSO - O interceptor lê o token do localStorage, não o "cria".
- c) VERDADEIRO - O interceptor adiciona o header x-access-token com o valor do token.
- d) FALSO - O acesso a rotas é gerido pelos Route Guards (CanActivate), não pelos Interceptors.

Parte 2 - Verdadeiro ou Falso

V/F 1. "O objeto XMLHttpRequest em JavaScript permite obter informação de um servidor sem o uso de formulários e sem a necessidade de fazer reload a uma página da internet."

Resposta:

VERDADEIRO. Esta é a definição do conceito de AJAX. Permite comunicar com o servidor de forma assíncrona sem recarregar a página.

V/F 2. "Um formulário para login de utilizadores pode utilizar o método GET para enviar informação de autenticação para o servidor."

Resposta:

FALSO. O método GET envia informação visível no URL. Enviar passwords via GET compromete gravemente a segurança. Usar SEMPRE POST com HTTPS.

V/F 3. "As aplicações Angular executam no runtime NodeJS."

Resposta:

FALSO. As aplicações Angular são client-side, executando diretamente no browser do utilizador. O Node.js é utilizado apenas como ferramenta de desenvolvimento.

V/F 4. "Um serviço REST com o método PUT não deve permitir o envio de informação de um cliente para o servidor."

Resposta:

FALSO. O método PUT é utilizado especificamente para Update (atualizar) recursos. Ele envia informação nova/atualizada no body do pedido HTTP.

Parte 3 - Resposta Aberta

Aberta 1. Descreva o significado dos conceitos backend e frontend no desenvolvimento de uma aplicação web. (0.5 val)

Resposta:

Frontend (Client-side): A parte visual da aplicação que corre no browser do utilizador (interface). Desenvolvido com HTML, CSS, JavaScript (e frameworks como Angular).

Backend (Server-side): A parte que corre num servidor remoto. Responsável pela lógica de negócio, comunicação com Bases de Dados, processamento seguro de autenticações.

Aberta 2. No desenvolvimento web indique o que entende pelo termo MEAN stack. (0.5 val)

Resposta:

É uma arquitetura de desenvolvimento de software baseada em JavaScript full-stack:

M = MongoDB (Base de dados NoSQL), E = ExpressJS (Framework web backend), A = Angular (Framework frontend), N = Node.js (Runtime de execução no servidor)

Aberta 3. As operações CRUD estão relacionadas com o desenvolvimento de vários componentes web. Indique de que se tratam estas operações e 2 casos de uso. (1 val)

Resposta:

CRUD: Create (Criar), Read (Ler), Update (Atualizar), Delete (Apagar).

1. Num API REST: Mapeiam-se para os métodos HTTP - POST (Create), GET (Read), PUT (Update), DELETE (Delete).

2. Na Base de Dados (Mongoose): save() (Create), find() (Read), findByIdAndUpdate() (Update), findByIdAndDelete() (Delete).

Aberta 4. Considere o padrão de software MVC. Descreva em que consiste este padrão e como o pode aplicar numa aplicação que utiliza ExpressJS. (1.5 val)

Resposta:

Model: Gere a estrutura dos dados e interage com a Base de Dados. No Express, ficheiros com schemas do Mongoose na pasta models/.

View: A parte visual enviada ao cliente. No Express, template engines como EJS (na pasta views/) para gerar HTML dinamico.

Controller: O intermediario que liga Views aos Models. No Express, ficheiros na pasta controllers/ que processam pedidos (req, res).

Aberta 5. Considere a seguranca de aplicacoes web. Indique como podemos adicionar autenticao e autorizacao a uma aplicacao web que use frameworks para o frontend e backend. (3 val)

Resposta:

1. Autenticacao: O backend recebe (via POST) as credenciais. Se validas, assina um token JWT com `jwt.sign({ userId, role }, segredo)`. O Angular guarda o token em `localStorage`.
2. Frontend: HTTP Interceptor - injeta o token no header de cada pedido. Guard (`CanActivate`) - impede que utilizadores nao autenticados acedam a paginas restritas.
3. Backend: Middleware `jwt.verify()` antes das rotas protegidas. Se token invalido -> 401. Se sem permissao -> 403.

Aberta 6. Observe a figura (backend processa `app.post("/result")` com username e password). Escreva o conteudo do ficheiro `demo.html` de forma a ficar funcional. (1.5 val)

Resposta:

Pontos-chave: `action="/result"`, `method="POST"`, `name="username"` e `name="password"` para corresponder aos campos que o Express espera em `req.body`.

```
<!DOCTYPE html>
<html>
<head><title>Pagina de Login</title></head>
<body>
  <h2>Login</h2>
  <form action="/result" method="POST">
    <label>Username:</label>
    <input type="text" name="username" required><br>
    <label>Password:</label>
    <input type="password" name="password" required><br>
    <button type="submit">Entrar</button>
  </form>
</body>
</html>
```

EXAME 2024/2025

Parte 1 - Escolha Multipla

P1. Considere o uso da ferramenta NPM nos projetos web desenvolvidos durante o semestre. Indique todas as afirmacoes que indicam funcionalidades desta ferramenta:

Resposta:

Corretas: b, c

- a) FALSO - O NPM instala modulos/packages JavaScript (como o mongoose), mas nao instala nem configura a base de dados MongoDB.
- b) VERDADEIRO - `npm start` executa o script "start" definido no `package.json`.
- c) VERDADEIRO - O NPM e um gestor de packages para JavaScript.
- d) FALSO - Criar componentes Angular faz-se com o Angular CLI: `ng generate component nome`.

P2. Indique quais dos seguintes comandos podem ser executados numa aplicacao da framework Angular:

Resposta:

Corretas: a, b, c

- a) VERDADEIRO - `ng generate component my-page` e o comando Angular CLI para criar um novo componente.

- b) VERDADEIRO - npm start pode ser executado em qualquer projeto Node.js/Angular.
- c) VERDADEIRO - npm install mongoose --save instala o modulo mongoose.
- d) FALSO - O comando correto e ng new my-app. "npm new" NAO EXISTE!

P3. Indique as afirmacoes verdadeiras sobre os componentes do tipo servico em Angular:

Resposta:

Corretas: a, b, d

- a) VERDADEIRO - Servicos usam @Injectable() e sao injetados via DI em multiplos componentes.
- b) VERDADEIRO - Servicos sao singletons e podem guardar estado global.
- c) FALSO - Servicos NAO tem template HTML. Sao classes TypeScript puras.
- d) VERDADEIRO - Usando Observables (RxJS) e BehaviorSubject, servicos propagam mudancas automaticamente.

P4. Considerando a framework ExpressJS utilizada em Node.js, indique as afirmacoes verdadeiras:

Resposta:

Corretas: a, b, c

- a) VERDADEIRO - Express permite organizar aplicacoes com o padrao MVC usando JavaScript/Node.js.
- b) VERDADEIRO - Express suporta template engines como EJS, Pug, Mustache com res.render().
- c) VERDADEIRO - Com o modulo mongoose, podemos ligar Express ao MongoDB para operacoes CRUD.
- d) FALSO - Express e server-side. O codigo Express executa no servidor Node.js, NAO no browser.

P5. Indique todas as afirmacoes verdadeiras:

Resposta:

Corretas: d

- a) FALSO - Angular e client-side (frontend). Para fullstack precisa-se de backend.
- b) FALSO - HTML e uma linguagem de marcacao (markup), nao de formatacao.
- c) FALSO - Node.js nao e uma linguagem de programacao - e um runtime para JavaScript.
- d) VERDADEIRO - Angular permite reutilizacao de componentes.

P6. Considere o excerto HTML com 3 elementos usando id="mvp" e getElementById("mvp"). Indique as afirmacoes verdadeiras:

Resposta:

Corretas: b

- b) VERDADEIRO - getElementById("mvp") retorna o primeiro elemento com id="mvp". A sua cor e alterada para vermelho.
- c) FALSO - getElementById retorna apenas UM elemento (o primeiro encontrado).
- d) FALSO - Apenas o primeiro <p> fica vermelho, nao todo o texto.

P7. Considere o excerto de codigo de um middleware Express com JWT (jwt.verify + verificacao de decoded.role). Indique as afirmacoes verdadeiras:

Resposta:

Corretas: b

- a) FALSO - O codigo usa req, res, next - e middleware Express (backend), nao Angular.
- b) VERDADEIRO - Usa jwt.verify() para autenticao E autorizacao (verificar decoded.role).
- c) FALSO - E um middleware (tem next()). Quando autenticao passa, chama next().
- d) FALSO - JWT pode ser usado com qualquer tipo de aplicacao web.

P8. Observe o excerto de codigo (@Injectable, HttpClient, getItem(): Observable<Item>, addItem()). Indique as afirmacoes verdadeiras:

Resposta:

Corretas: b

- a) FALSO - O código usa @Injectable, HttpClient, Observable - tudo específico de Angular, não Express.
- b) VERDADEIRO - É um serviço Angular que consome uma API REST via HttpClient.
- c) FALSO - Retorna um Observable<Item>, que é assíncrono. O valor só é obtido com .subscribe().
- d) FALSO - Não se pode afirmar que existe BD.

Parte 2 - Verdadeiro ou Falso

V/F 1. "A validação de dados de input deve ser apenas realizada no frontend de uma aplicação web para garantir que só dados corretos chegam ao backend."

Resposta:

FALSO. A validação deve ser feita em AMBOS os lados - frontend (UX) E backend (segurança). O utilizador pode contornar o frontend. Confiar apenas no frontend é uma falha de segurança grave.

V/F 2. "CSS é considerada uma linguagem de programação pois permite formatar e realizar animações em páginas HTML."

Resposta:

FALSO. CSS é uma linguagem de estilos, não de programação. Não possui lógica de controlo geral (if/else), funções definidas pelo programador, nem manipulação de dados.

V/F 3. "O módulo JWT não é adequado para garantir autenticação e autorização em páginas web codificadas com a framework ExpressJS e o template engine EJS."

Resposta:

FALSO. JWT é adequado e pode ser usado com ExpressJS e EJS. JWT funciona como middleware no Express. Não depende do template engine.

V/F 4. "É possível utilizar a base de dados MongoDB no frontoffice de uma aplicação web para guardar dados da aplicação de forma persistente."

Resposta:

FALSO. MongoDB executa no servidor (backend), não no frontend. Para persistência no frontend usa-se localStorage.

Parte 3 - Resposta Aberta

Aberta 1. Indique o que entende pelo conceito cliente-servidor em aplicações web. (1 val)

Resposta:

O modelo cliente-servidor é uma arquitetura de software com dois intervenientes:

Cliente: O browser do utilizador, que executa HTML, CSS e JavaScript (frontend). Inicia a comunicação enviando pedidos HTTP.

Servidor: Aplicação remota (ex: Node.js + Express) que recebe pedidos, processa lógica de negócio, acede a bases de dados, e devolve respostas.

Na MEAN Stack: Angular = cliente, Node.js/Express = servidor.

Aberta 2. Explique o que são serviços REST no âmbito do desenvolvimento de aplicações para a web. (2 val)

Resposta:

REST (Representational State Transfer) é um padrão arquitetural que define restrições para a criação de web services baseados em HTTP.

GET -> Read, POST -> Create, PUT -> Update, DELETE -> Delete.

Propriedades REST: Stateless, Cacheable, Interface uniforme, Cliente-servidor, Sistema em camadas.

Respostas em JSON (ou XML). Para testes usa-se Postman, para documentação Swagger/OpenAPI, para segurança CORS e JWT.

Aberta 3.1. Identifique e corrija os erros presentes no código do Click Counter. (1 val)

Resposta:

Erro 1: numeroCliques nunca e declarado nem inicializado -> var numeroCliques = 0;
Erro 2: getElementById(counter) -> Deve ser string: getElementById("contador")
Erro 3: O botao nao tem evento associado -> Adicionar onclick="contarClique()" ao botao

```
<button onclick="contarClique()">Click Me</button>
<script>
  var numeroCliques = 0;
  function contarClique() {
    numeroCliques++;
    document.getElementById("contador").textContent = numeroCliques;
  }
</script>
```

Aberta 3.2. Como podemos melhorar a pagina para que a contagem de cliques continue e nao seja reiniciada ao reabrir a pagina? (1 val)

Resposta:

Usar localStorage para guardar o valor do contador de forma persistente. O localStorage persiste entre sessoes (sobrevive ao fechar o browser), ao contrario do sessionStorage.

```
<script>
  var numeroCliques = parseInt(localStorage.getItem("contadorCliques")) || 0;
  document.getElementById("contador").textContent = numeroCliques;
  function contarClique() {
    numeroCliques++;
    document.getElementById("contador").textContent = numeroCliques;
    localStorage.setItem("contadorCliques", numeroCliques);
  }
</script>
```

Aberta 4. Comente a frase: "Os componentes criados na framework Angular cumprem o padrao MVC". (1.5 val)

Resposta:

A afirmacao e verdadeira. Os componentes Angular cumprem o padrao MVC:
Model: Classes/Interfaces e Servicos (@Injectable) - ficheiros *.model.ts, *.service.ts
View: Template HTML do componente - *.component.html + *.component.css
Controller: Classe TypeScript (@Component) - *.component.ts
O component.ts (Controller) contem a logica. O component.html (View) apresenta dados via data binding (interpolation, property binding, event binding, two-way binding).

Aberta 5. Quais as funcoes dos componentes Guard e Intercept numa aplicacao escrita com a framework Angular? (1.5 val)

Resposta:

Guards (Guardas de Rota): Implementam CanActivate. Controlam o acesso a rotas/paginas. Se retornam true, navegacao prossegue; false, bloqueia. Ex: Verificar se existe token em localStorage; se nao, redirecionar para /login.
Interceptors (Interceptadores HTTP): Implementam HttpInterceptor. Modificam pedidos HTTP antes de serem enviados. Executados automaticamente em todos os pedidos do HttpClient. Ex: Adicionar token JWT ao header.
Guards protegem rotas (autorizacao de navegacao). Interceptors modificam pedidos HTTP (autenticacao). Ambos sao fundamentais para autenticacao/autorizacao em Angular.

Dica final: Pratica estes exames em condicoes de tempo real (1h30min). Faz cada exame completo sem consultar as respostas. Depois corrige e anota os pontos fracos.